**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Direccion** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 1 |

**TEMA: Fundamentos del Paradigma Orientado a Objetos**

**1) Definición**

La Programación Orientada a Objetos (POO) es un paradigma de programación que estructura el diseño de software alrededor de entidades denominadas objetos, que son modelos abstractos de elementos del mundo real. A diferencia de la programación estructurada, que se centra en los procedimientos y la lógica, la POO une los datos (qué es) y las funciones (qué hace) que manipulan esos datos dentro de una sola unidad, el objeto.

- Contraste con Programación Estructurada: La programación estructurada organiza el código en funciones que operan sobre datos globales o pasados por parámetro, mientras que la POO encapsula los datos con las funciones que los afectan (métodos), promoviendo una organización modular y más cercana a la manera en que se percibe el mundo.

---

## 2) Conceptos Clave

Estos son los conceptos esenciales que permiten el modelado de la realidad en POO:

- Clase: Actúa como la plantilla o el plano de construcción que define las características generales (atributos) y las capacidades (métodos) comunes a una familia de objetos.
- Objeto: Es una instancia concreta y específica de una clase. Representa una entidad única con un estado particular, determinado por los valores de sus atributos.
- Atributos: Son las propiedades o datos que definen el estado de un objeto en un momento dado.
- Métodos: Representan las acciones o comportamientos que un objeto puede ejecutar. Son las operaciones que se realizan sobre los atributos del objeto, definiendo su funcionalidad.
- Encapsulamiento: Es el principio de agrupar los atributos y los métodos en una única unidad (el objeto), y de proteger el estado interno del objeto. Esto significa que los datos (atributos) solo se pueden manipular a través de los métodos definidos por la clase, garantizando el control sobre su acceso y modificación.
- Ciclo de Vida: Los objetos tienen un ciclo que incluye su creación (instanciación), su uso (invocación de métodos) y su destrucción (liberación de recursos de memoria).

---

## 3) Ejemplos en Texto

Para comprender los fundamentos de la POO, se puede tomar como ejemplo el diseño y funcionamiento de un Sistema de Semáforos Inteligentes en una ciudad:
Modelado de un Semáforo

1. Clase: La Clase es Semáforo. Esta es la definición abstracta de *cualquier* semáforo. Define que todos los semáforos deben tener luces (roja, amarilla, verde) y la capacidad de cambiar de estado.

2. Objetos: Los Objetos son las instancias específicas de esa clase en el mundo real:
    - semaforoCrucePrincipal (el que está en la esquina A).
    - semaforoSalidaColegio (el que está en la esquina B).
3. Atributos (Datos): Los atributos almacenan el estado de cada objeto:
    - colorActual: (Puede ser "Rojo", "Amarillo" o "Verde").
    - tiempoRestante: (El tiempo que le queda en el color actual, ej. 30 segundos).
4. Métodos (Comportamiento): Los métodos permiten cambiar o consultar el estado de cada objeto:
    - cambiarARojo(): Modifica el atributo colorActual a "Rojo".
    - mostrarTiempo(): Devuelve el valor del atributo tiempoRestante.
5. Encapsulamiento: El sistema no permite modificar el color directamente. Para cambiar el color, se debe invocar el método, por ejemplo, cambiarARojo(). Este método contiene la lógica de seguridad (asegurarse de que una luz no cambie de "Rojo" a "Verde" sin pasar por "Amarillo"), protegiendo así el estado interno del objeto y asegurando que las reglas del sistema se cumplan.

---

**4) Conclusiones**

La adopción del paradigma Orientado a Objetos es fundamental en el desarrollo de software moderno, ya que proporciona una metodología de diseño que promueve la robustez y la escalabilidad del código. Los Fundamentos de POO, basados en la Clase como molde, el Objeto como instancia y el Encapsulamiento como mecanismo de protección, generan un código que es más fácil de mantener y reutilizar. Esta forma de pensar permite a los desarrolladores modelar sistemas complejos de forma modular, donde cada componente (objeto) tiene responsabilidades claras, facilitando la comprensión y la colaboración en proyectos grandes.

**5) Bibliografía**


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Direccion** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 2 |

**Tema: Principios de Diseño en POO**

**1) Definición**

Los Principios de Diseño en POO son el conjunto de conceptos avanzados (Herencia, Polimorfismo y Relaciones entre Clases) que van más allá del simple uso de clases y objetos. Estos principios, a menudo resumidos en el acrónimo PIE (Polimorfismo, Herencia y Encapsulamiento, siendo este último visto en la unidad 1), permiten crear sistemas modulares, fáciles de extender y de bajo acoplamiento. La aplicación adecuada de estos principios conduce a un diseño de software que es más maleable ante futuros cambios.

---

## 2) Conceptos Clave

A. Introducción a la Herencia

La Herencia es un mecanismo que permite a una clase (denominada subclase o clase hija) adquirir o "heredar" los atributos y métodos de otra clase (denominada superclase o clase padre).

- Creación de Jerarquías Simples: La herencia organiza las clases en una estructura jerárquica (tipo árbol) que representa relaciones de tipo "es un". Por ejemplo, un Perro es un Animal.
- Reutilización de Código: Su principal beneficio es la reutilización, ya que las características comunes se definen una sola vez en la superclase y se comparten automáticamente con todas las subclases.

B. Concepto de Polimorfismo Básico

El término Polimorfismo proviene del griego y significa "muchas formas". Este principio permite que un mismo mensaje (la llamada a un método) sea interpretado de distintas maneras por diferentes objetos.

- Métodos Sobrescritos (Overriding): Es la forma más básica de polimorfismo. Ocurre cuando una subclase redefine la implementación de un método que ya ha sido heredado de su superclase. Esto permite que, aunque el nombre del método sea idéntico, su comportamiento sea específico para el tipo de objeto que lo ejecuta.

C. Relaciones entre Clases

Las clases no existen aisladas; interactúan entre sí. Estas interacciones se denominan relaciones:

- Asociación: Describe una relación estructural de "tiene un" entre dos o más objetos. Las clases asociadas son independientes y pueden existir sin la otra (Ej. Un Profesor está asociado con un Curso).
- Agregación: Es una forma especial de asociación que representa una relación de "todo-parte" o pertenencia. Las partes (objetos) pueden existir de forma independiente del todo (Ej. Un Equipo de Fútbol (todo) tiene Jugadores (partes), y un Jugador puede existir aunque el Equipo se disuelva).
- Composición: Es la relación más fuerte de "todo-parte". La existencia de las partes está ligada a la existencia del todo. Si el objeto principal (el todo) es destruido, las partes que lo componen

también dejan de existir (Ej. Un Motor es una parte de un Automóvil; si el Automóvil se destruye, el Motor ya no tiene el mismo contexto de existencia en el sistema).

---

## 3) Ejemplos en Texto

Consideremos el diseño de un sistema de nómina para una Empresa de Servicios:
Modelado de Personal

1. Herencia (Relación "Es un"):
    - Definimos una Clase Padre abstracta: Empleado. Esta clase contiene atributos comunes a todo el personal (nombre, salarioBase, antiguedad) y un método genérico: calcularSalario().
    - Creamos dos Clases Hijas: Gerente y Operario. Ambos son un Empleado, por lo que heredan todos sus atributos.
2. Polimorfismo (Métodos Sobrescritos):
    - El método calcularSalario() de la clase Empleado simplemente devuelve el salarioBase.
    - La clase Gerente sobrescribe el método calcularSalario() para incluir una bonificación de 10% por rendimiento.
    - La clase Operario sobrescribe el método calcularSalario() para restar un 5% por deducciones obligatorias.
    - Resultado: Al llamar a la función personal.calcularSalario() sobre un objeto Gerente o un objeto Operario, el sistema ejecuta la versión del método que es específica para ese tipo de empleado, utilizando una única interfaz (el mismo nombre de método) para obtener resultados diferentes.
3. Relaciones (Asociación y Composición):
    - Composición (Fuerte): Definimos la clase Oficina. Cada objeto Oficina está compuesto por un objeto Dirección (calle, ciudad, código postal). Si el objeto Oficina es eliminado del sistema, la Dirección asociada ya no tiene sentido por sí misma.
    - Asociación (Débil): La clase Empleado está asociada a la clase Proyecto. Un empleado puede ser asignado a varios proyectos a lo largo del tiempo, y tanto el Empleado como el Proyecto pueden existir de forma independiente.

## 4) Conclusiones

Los principios de diseño en POO son cruciales para superar las limitaciones de la programación simple basada en objetos. La Herencia permite la creación de jerarquías coherentes que facilitan la reutilización, mientras que el Polimorfismo dota al código de la capacidad de responder dinámicamente según el tipo de objeto en tiempo de ejecución. Finalmente, el modelado de Relaciones (Asociación, Composición, Agregación) asegura que las interacciones entre componentes del sistema sean claras y lógicas. Dominar estos principios es esencial para desarrollar aplicaciones flexibles, de fácil mantenimiento y que puedan adaptarse a los requisitos cambiantes del negocio sin reescribir grandes porciones de código.

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Direccion** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 3 |

**Tema: Introducción al Entorno de Desarrollo**

**1) Definición**

El Entorno de Desarrollo se refiere al conjunto integrado de *software* y herramientas que un programador utiliza para la creación eficiente de aplicaciones. Un entorno moderno va más allá de un simple editor de texto, proporcionando funcionalidades para la edición de código, la depuración (debugging), la compilación, la automatización de tareas y, crucialmente, la gestión de versiones del *software*.

---

## 2) Conceptos Clave

A. Entorno de Desarrollo Integrado (IDE)

Un IDE (*Integrated Development Environment*) es una aplicación de *software* que consolida las funcionalidades esenciales para la programación en una única interfaz gráfica.

- Uso de IDEs Modernos: Los IDEs como IntelliJ, NetBeans o Visual Studio Code (VS Code, aunque es más un editor de código avanzado, funciona como un IDE con extensiones) son fundamentales. Ofrecen *features* como el resaltado de sintaxis, la autocompletación inteligente (IntelliSense) y la navegación rápida por el código, aumentando drásticamente la productividad.

B. Flujo de Proyecto

El proceso de llevar el código escrito a una aplicación ejecutable implica varios pasos cruciales:

- Creación de Proyectos: Los IDEs permiten crear una estructura organizada de archivos y carpetas para el código fuente, librerías y recursos, asegurando que el proyecto tenga una configuración predeterminada para su compilación y ejecución.
- Compilación: Es el proceso que traduce el código fuente escrito en un lenguaje de alto nivel (como Java o C++) a código de máquina o *bytecode* (código intermedio) que la computadora puede entender y ejecutar.
- Ejecución: Es la fase donde el código compilado se carga en la memoria del sistema operativo y comienza a realizar las tareas para las que fue diseñado.

C. Control de Versiones con Git

Git es el estándar de la industria para los Sistemas de Control de Versiones Distribuidos (*DVCS*). Permite rastrear los cambios en el código fuente a lo largo del tiempo, facilitando la colaboración entre desarrolladores y permitiendo revertir el código a estados anteriores de manera segura.

- Integración con IDEs: La mayoría de los IDEs se integran directamente con Git, permitiendo operaciones como *commit* (guardar un punto de control del trabajo), *push* (enviar cambios a un repositorio remoto), y *pull* (descargar cambios de otros colaboradores) sin salir del entorno de desarrollo.

---

### 3) Ejemplos en Texto (Aplicación en la Vida Real)

Imaginemos un equipo de desarrollo que crea una aplicación de banca móvil utilizando el paradigma POO.

1. Uso del IDE (IntelliJ): Un desarrollador utiliza IntelliJ para escribir la clase CuentaBancaria. Mientras escribe, el IDE detecta automáticamente errores de sintaxis y sugiere métodos y atributos de la clase Cliente gracias a su autocompletación inteligente. Esto evita errores manuales y acelera el proceso de codificación.
2. Creación y Compilación: El desarrollador crea una nueva funcionalidad (transferirFondos) y la incorpora al proyecto. Al presionar el botón de "Compilar" en el IDE, el *software* toma el código Java de la clase CuentaBancaria, lo traduce a *bytecode*, y lo empaqueta en un archivo .jar listo para ser ejecutado en la máquina virtual Java.
3. Control de Versiones (Git): Antes de terminar el día, el desarrollador necesita guardar su progreso. Utiliza la herramienta Git integrada en el IDE para:
    - Realizar un *Commit*: Guarda una instantánea local de los cambios con un mensaje claro ("Implementada la lógica de validación para transferencias").
    - Realizar un *Push*: Envía los cambios al repositorio central (como GitHub o GitLab). Esto permite que el segundo desarrollador, que está trabajando en la clase Notificación, pueda descargar (hacer un *pull*) el código actualizado de CuentaBancaria e integrar sin problemas la nueva funcionalidad.

Este flujo asegura que todo el equipo trabaje con la versión más reciente del código, minimizando conflictos y permitiendo una revisión ordenada de la historia del proyecto.

---

## 4) Conclusiones

La Introducción al Entorno de Desarrollo no es solo aprender a usar una herramienta, sino a adoptar un flujo de trabajo profesional. El dominio de un IDE moderno optimiza la productividad y reduce los errores, mientras que la integración con Git transforma la programación individual en un proceso de colaboración estructurado y seguro. Estos conocimientos son tan vitales como el dominio de los principios POO, ya que permiten aplicar la teoría en proyectos reales que requieren de trazabilidad, trabajo en equipo y gestión de cambios eficiente.

---

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Direccion** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 4 |

**Tema: Estructuras de Datos Iniciales en POO**

**1) Definición**

Las Estructuras de Datos Iniciales en POO se refieren a los mecanismos básicos proporcionados por el lenguaje de programación para guardar y organizar la información. Estas estructuras son esenciales porque permiten manejar colecciones de datos de manera eficiente, ya sea que se trate de información simple (como números y caracteres, manejados por tipos primitivos) o de colecciones más complejas de objetos (como *arrays* y listas).

---

## 2) Conceptos Clave

A. Tipos de Datos Primitivos y Objetos

En POO, se distinguen dos grandes categorías para manejar datos:

- Tipos de Datos Primitivos: Son los tipos de datos fundamentales y más simples que manejan valores únicos y atómicos (ej. enteros, números decimales, booleanos y caracteres). Suelen almacenarse directamente en la pila de memoria (*stack*) y no poseen métodos ni propiedades.
- Objetos: A diferencia de los primitivos, son instancias de clases. Un objeto (o una referencia a un objeto) se almacena generalmente en la memoria *heap*, y está compuesto por múltiples atributos y, crucialmente, posee métodos o comportamientos asociados.

B. Arreglos (Arrays)

El Arreglo o *Array* es la estructura de datos más básica para colecciones y se caracteriza por ser una secuencia de elementos del mismo tipo, almacenados en posiciones de memoria contiguas.

- Declaración y Uso: Los *arrays* requieren que su tamaño se defina al momento de la inicialización, lo que significa que son de tamaño fijo.
- Tipos:
    - Unidimensionales: Son listas de elementos (Ej. Una fila).
    - Bidimensionales: Son tablas o matrices que requieren dos índices (fila y columna) para acceder a un elemento (Ej. Una cuadrícula o un tablero de ajedrez).

C. Listas Básicas

Las Listas son estructuras de datos que también almacenan una colección de elementos, pero a diferencia de los *arrays*, son de tamaño dinámico.

- Diferencias con Arreglos: Su principal ventaja es que pueden crecer o reducirse a medida que el programa lo requiere, lo que las hace más flexibles que los *arrays* en situaciones donde no se conoce el número exacto de elementos. Permiten inserciones y eliminaciones más sencillas que los *arrays*.

D. Iteradores

Un Iterador es un objeto que permite recorrer los elementos de una colección (como un *array* o una lista) de manera secuencial, sin exponer la representación subyacente de la colección.

- Concepto: Proporcionan una forma estandarizada de acceso a los elementos de una colección, con métodos comunes como hasNext() (para verificar si hay más elementos) y next() (para obtener el siguiente elemento).

---

**3) Ejemplos en Texto**

Consideremos el diseño de un sistema de gestión de inventario para un Supermercado:

1. Tipos Primitivos y Objetos:
    - Un producto individual (Manzana) es un Objeto, instancia de la clase Producto, que tiene atributos como nombre, código (un número entero, que es un tipo primitivo) y un método vender().
    - El precio unitario ($1.50) puede ser representado por un tipo de dato primitivo (float o double), ya que es un valor simple.
2. Arreglos Unidimensionales (Tamaño Fijo):
    - Al comienzo del día, el sistema de caja necesita almacenar los códigos de barras de los 10 cajeros que están trabajando. Como el número de cajas es fijo (10), se usa un Arreglo Unidimensional de 10 posiciones para almacenar estos códigos.
    - *Ventaja:* Acceder al código del Cajero 3 es instantáneo por su índice (cajeros[2]).
3. Listas Básicas (Tamaño Dinámico):
    - El sistema necesita registrar todos los productos que están Agotados en las últimas 24 horas. El número de productos agotados es impredecible (puede ser 5 un día y 50 al siguiente). Para esto, se utiliza una Lista, que puede crecer dinámicamente según se reporten más productos faltantes.
    - *Ventaja:* Se pueden añadir o remover productos de la lista sin tener que preocuparse por la capacidad inicial de almacenamiento.
4. Iteradores:
    - Para generar el Reporte Diario de Agotados, el sistema debe recorrer la Lista de Agotados. En lugar de depender de la forma interna en que la lista almacena los datos, se utiliza un

Iterador para asegurar que cada producto agotado sea visitado una sola vez y añadido al reporte, sin importar si la lista es un *array* o una estructura más compleja.

---

## 4) Conclusiones

Las Estructuras de Datos Iniciales en POO son la base sobre la que se construye la lógica de almacenamiento de cualquier aplicación. La comprensión de los tipos primitivos y la diferenciación con el concepto de objeto es fundamental. La elección entre estructuras estáticas como los Arreglos y estructuras dinámicas como las Listas es una decisión crítica de diseño que impacta directamente la eficiencia y la flexibilidad del sistema. Finalmente, el concepto de Iterador proporciona un mecanismo elegante y seguro para interactuar con las colecciones, manteniendo la abstracción y la modularidad que caracterizan al paradigma POO.

---

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Direccion** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 5 |

**Tema: Introducción a la Modularidad y Reutilización**

**1) Definición**

La Modularidad y Reutilización son pilares del diseño de *software* en POO que buscan gestionar la complejidad del sistema dividiéndolo en componentes discretos (módulos) y promoviendo el uso de código ya existente en lugar de escribirlo de nuevo.

- Modularidad: Implica descomponer un programa en partes funcionales e independientes, como clases, paquetes o bibliotecas, que se comunican a través de interfaces bien definidas. Un módulo debe tener una alta cohesión (todos sus elementos están relacionados) y un bajo acoplamiento (su dependencia con otros módulos es mínima).
- Reutilización: Es la capacidad de utilizar las partes (clases, métodos, paquetes) ya creadas y probadas en nuevos proyectos o

en diferentes secciones del mismo proyecto, lo cual se logra principalmente a través de la herencia y la modularización.

---

## 2) Conceptos Clave

A. Concepto de Módulos y Paquetes en POO

Los Paquetes (*Packages* en Java, *Namespaces* en C# o C++) son el mecanismo principal de la POO para lograr la modularidad y el control de acceso a gran escala.

- Función: Los paquetes agrupan clases relacionadas por funcionalidad (Ej. com.banco.transacciones o org.utilidades.fechas). Esto evita conflictos de nombres entre clases que podrían llamarse igual en diferentes contextos y define un ámbito de visibilidad para las clases.
- Módulos: En lenguajes de programación modernos, un módulo puede considerarse una unidad de código más grande que un paquete, que contiene uno o más paquetes y está diseñado para ser la unidad fundamental de distribución y despliegue del *software*.

B. Creación y Uso de Bibliotecas

Una Biblioteca (*Library* o *JAR* en Java) es un conjunto de código precompilado (clases y paquetes) que contiene funcionalidades específicas y que está listo para ser incluido y utilizado por otro proyecto.

- Bibliotecas Propias: Un desarrollador puede tomar un conjunto de clases que ha creado (Ej. un motor de cálculo de impuestos) y empaquetarlas como una biblioteca. Esto permite que otros proyectos dentro de la misma organización la reutilicen sin tener acceso al código fuente interno, aplicando el principio de encapsulamiento a nivel de módulo.

C. Importación y Uso de Bibliotecas Estándar

Casi todos los lenguajes de POO vienen con una Biblioteca Estándar (API) que proporciona un amplio conjunto de funcionalidades básicas (Ej. manejo de archivos, operaciones de red, colecciones avanzadas).

- Importación: Para usar estas funcionalidades, el desarrollador simplemente las importa o las incluye en su código, lo que ilustra el concepto de reutilización: no es necesario escribir, por ejemplo, el código para leer un archivo, ya que el lenguaje lo proporciona.

D. Modularidad para Desarrollo Escalable

La modularidad es clave para la escalabilidad. Al dividir el sistema en componentes independientes, las mejoras o modificaciones pueden realizarse en un solo módulo sin afectar el funcionamiento de otros módulos, lo que reduce el riesgo y el tiempo de mantenimiento.

---

**3) Ejemplos en Texto**

Consideremos el desarrollo de un Sistema de Gestión Académica para una universidad:

1. Modularización con Paquetes:
    - El equipo de desarrollo organiza el sistema en los siguientes paquetes (módulos):
        - edu.universidad.matriculas: Contiene clases como Estudiante, Curso y ProcesoMatricula.
        - edu.universidad.calificaciones: Contiene clases como Nota, RegistroNotas y CalculadoraPonderada.
        - edu.universidad.usuarios: Contiene clases de gestión de acceso como Usuario, Rol y Autenticador.
2. Reutilización mediante Bibliotecas Estándar y Propias:
    - En el paquete calificaciones, el método para calcular el promedio de un estudiante utiliza una función para redondear el resultado. En lugar de escribir esa función, el desarrollador importa y reutiliza la clase DecimalFormat de la biblioteca estándar del lenguaje.
    - El equipo de desarrollo de la universidad ya había creado una biblioteca propia llamada UtilidadesGenerales que contiene una clase para validar el formato de correos institucionales. El paquete matriculas simplemente importa esta biblioteca y reutiliza el método de validación.
3. Impacto en la Escalabilidad:
    - Si la universidad decide cambiar su política de calificación (por ejemplo, cambiar el algoritmo de ponderación), solo se necesita modificar y probar el código dentro del paquete edu.universidad.calificaciones. El paquete edu.universidad.matriculas (que solo llama al método calcularPromedio) no necesita ser modificado, ya que la

interfaz del método no cambió, demostrando un bajo acoplamiento y un diseño escalable.

---

## 4) Conclusiones

La Modularidad y la Reutilización son prácticas de diseño de software que aseguran la eficiencia y la longevidad de los proyectos POO. Al usar paquetes para organizar el código y bibliotecas para encapsular funcionalidades probadas, los desarrolladores minimizan la duplicación de código, lo que reduce la probabilidad de errores y facilita el mantenimiento. Este enfoque estructurado es crucial para la escalabilidad, permitiendo que grandes equipos trabajen en diferentes partes del sistema simultáneamente con mínimas interferencias, sentando las bases para sistemas empresariales robustos.

---

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Dirección** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 6 |

**Tema: Manejo Básico de Excepciones**

**1) Definición**

El Manejo de Excepciones es un mecanismo en POO que permite gestionar y responder a situaciones inesperadas o errores (excepciones) que surgen en tiempo de ejecución. Una excepción representa un evento que interrumpe el flujo normal de las instrucciones del programa. En lugar de permitir que el programa termine inesperadamente (lo que se conoce como *crashear*), el manejo de excepciones proporciona una estructura para capturar el error, procesarlo y permitir que el programa continúe su ejecución de manera controlada.

- Utilidad: Las excepciones son esenciales para la calidad del *software*, ya que permiten separar la lógica de negocio del código de manejo de errores, haciendo que el código principal sea más limpio y legible.

---

## 2) Conceptos Clave

A. Definición de Excepciones y su Utilidad

Una Excepción es un objeto que se crea cuando ocurre un evento anómalo. Los lenguajes de POO tienen una jerarquía de clases de excepciones predefinidas (Ej. NullPointerException, IOException, ArithmeticException) que clasifican el tipo de error.

- Utilidad: Son necesarias para manejar situaciones que no son errores de sintaxis (que detecta el compilador), sino fallos lógicos o de entorno (que ocurren al ejecutar el programa), como intentar dividir por cero o acceder a un archivo inexistente.

B. Bloques try-catch-finally

Los bloques try-catch-finally son la estructura sintáctica fundamental para implementar el manejo de excepciones:

- try: Contiene el código que puede generar la excepción. El sistema supervisa la ejecución de este bloque. Si no ocurre ninguna excepción, el bloque catch se ignora.
- catch: Se ejecuta solo si ocurre una excepción dentro del bloque try. Este bloque recibe el objeto excepción y contiene el código de recuperación, registro o notificación del error.
- finally: Contiene código que se ejecutará siempre, haya ocurrido una excepción o no. Se utiliza generalmente para liberar recursos esenciales (Ej. cerrar una conexión de base de datos o un archivo).

C. Creación de Excepciones Personalizadas

Los desarrolladores pueden crear sus propias clases de excepciones.

- Propósito: Se utilizan para modelar errores específicos de la lógica del negocio que no están cubiertos por las excepciones estándar del lenguaje (Ej. SaldoInsuficienteException en un sistema bancario). Esto facilita que el código que llama identifique y maneje el error con mayor precisión.

D. Importancia del Manejo de Errores en Aplicaciones Reales

Un manejo de excepciones deficiente puede llevar a la pérdida de datos, fallos de seguridad o una mala experiencia para el usuario. Un buen manejo de errores garantiza la integridad de la aplicación y ofrece mensajes de error descriptivos y acciones de recuperación al usuario.

---

## 3) Ejemplos en Texto

Consideremos una aplicación de Cálculo de Préstamos Bancarios:

1. Bloque try (La Operación Crítica):
    - El usuario ingresa el monto del préstamo y la tasa de interés. El código crucial para el cálculo matemático se coloca en un bloque try.
2. Manejo de Excepciones Estándar (catch):
    - Situación Anómala: El usuario, por error, ingresa un valor de "0" como el número de meses para pagar el préstamo. Esto provoca una división por cero, lo cual genera una ArithmeticException.
    - Respuesta Controlada: El programa captura esta excepción en un bloque catch específico. En lugar de detenerse, muestra un mensaje amigable al usuario: "Error: El plazo de pago no puede ser cero. Por favor, ingrese un valor positivo."
3. Excepción Personalizada:
    - El sistema tiene una regla de negocio: el monto del préstamo no puede exceder el 10% del salario del solicitante.
    - Si el usuario pide un monto excesivo, el programa lanza (o *throws*) una Excepción Personalizada: MontoExcesivoException.
    - Esta excepción se captura en un bloque catch que informa al usuario: "Su solicitud excede el límite permitido para su nivel de ingresos. Intente con un monto menor."
4. Bloque finally (Liberación de Recursos):
    - Independientemente de si el cálculo del préstamo tuvo éxito o falló por un error (división por cero, monto excesivo, etc.), se utiliza un bloque finally para asegurarse de que la conexión con la base de datos se cierre correctamente.

---

## 4) Conclusiones

El Manejo de Excepciones es un aspecto obligatorio en la programación de sistemas robustos y confiables. Al emplear la estructura try-catch-finally, los desarrolladores aseguran que las fallas esperadas e inesperadas se aborden sin comprometer la estabilidad del sistema. Este

mecanismo no solo previene caídas del programa, sino que también mejora la experiencia del usuario al proporcionarle retroalimentación útil sobre lo que salió mal, y permite mantener la modularidad al separar la lógica de error del flujo normal del negocio.

---

**5) Bibliografía**


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Dirección** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 7 |

**Tema: Programación Orientada a Objetos y el Ciclo de Vida del Software**

**1) Definición**

La Programación Orientada a Objetos y el Ciclo de Vida del Software (CVDS) estudian cómo los principios de la POO (clases, objetos, herencia, polimorfismo) se aplican y se integran en cada una de las fases del desarrollo de un sistema. El CVDS proporciona un marco de trabajo que garantiza la calidad y la trazabilidad, mientras que la POO ofrece el paradigma de modelado ideal para llevar esos requisitos a la implementación.

---

## 2) Conceptos Clave

A. Introducción al Desarrollo Basado en Requisitos Simples

El CVDS comienza con la fase de Análisis de Requisitos, donde se identifica qué debe hacer el *software*. En el contexto POO, esto implica:

- Identificación de Entidades: Reconocer las entidades del mundo real que se modelarán como Clases y Objetos (Ej. En un sistema de biblioteca, las entidades son *Libro*, *Usuario*, *Préstamo*).
- Requisitos Funcionales: Definir las operaciones que estos objetos deben realizar (los Métodos).
- Requisitos No Funcionales: Definir la calidad, seguridad y rendimiento del sistema.

B. Creación de Diagramas Básicos UML

El Lenguaje Unificado de Modelado (UML) es la notación gráfica estándar utilizada para visualizar, especificar, construir y documentar los artefactos de un sistema orientado a objetos. Es la herramienta clave en la fase de Diseño del CVDS.

- Diagramas de Casos de Uso: Muestran la interacción entre el sistema y los actores (usuarios o sistemas externos), describiendo las funcionalidades (*qué* hace el sistema).
- Diagramas de Clases: Muestran la estructura estática del sistema: las Clases, sus Atributos y Métodos, y las Relaciones entre ellas (asociación, herencia, composición) .

C. Diseño y Desarrollo de Pequeños Proyectos en POO

La fase de Implementación toma los diseños UML y los traduce a código fuente (POO).

- Trazabilidad: Una buena práctica en esta fase es asegurar la trazabilidad entre los elementos del diseño (las clases del diagrama) y el código final (las clases del lenguaje de programación), asegurando que el *software* cumpla con lo planificado.
- Desarrollo Iterativo: En metodologías modernas, el desarrollo no es lineal; se trabaja en ciclos pequeños (iteraciones) que entregan funcionalidades incrementales, probando y refactorizando continuamente el código orientado a objetos.

D. Importancia de la Documentación Técnica

La Documentación es una fase continua a lo largo del CVDS.

- Propósito: Proporciona un registro claro de cómo está diseñado y construido el sistema. En POO, esto incluye la documentación de la

interfaz pública de las clases y métodos (Ej. qué parámetros acepta un método y qué valor retorna), lo cual facilita el mantenimiento y la colaboración.

---

### 3) Ejemplos en Texto

Imaginemos que una empresa de *e-commerce* decide crear un Sistema de Carrito de Compras.

1. Análisis de Requisitos y Modelado (UML):
    - El equipo identifica la necesidad de gestionar Productos, Clientes y Órdenes.
    - Se crea un Diagrama de Casos de Uso donde el actor Cliente puede "Añadir Producto al Carrito" y "Realizar Pago".
    - Se diseña un Diagrama de Clases que muestra:
        - Una clase Producto con atributos (nombre, precio).
        - Una clase Carrito que tiene una relación de Composición con la clase ItemCarrito (porque un ítem solo existe dentro de un carrito).
        - Una clase TarjetaDeCredito que tiene una Asociación con el Cliente.
2. Implementación con POO:
    - El programador toma la clase Carrito del diseño UML y la codifica en Java o C++.
    - Para la funcionalidad "Realizar Pago", se invoca el método procesarPago() de la clase TarjetaDeCredito, asegurando que toda la lógica de validación y comunicación con la pasarela de pago esté encapsulada dentro de esa clase.
3. Documentación Continua:
    - A medida que se desarrolla el método procesarPago(monto), el desarrollador añade comentarios (*Javadoc* o similar) explicando que el método lanza una PagoFallidoException si el *token* de la tarjeta es inválido, lo cual es esencial para que otros desarrolladores puedan integrar la función correctamente.

---

## 4) Conclusiones

La integración de la Programación Orientada a Objetos con el Ciclo de Vida del Software es la mejor práctica para la ingeniería de *software*. El uso de UML en la fase de Diseño proporciona un lenguaje visual que une el análisis de requisitos con la implementación. Al aplicar la POO de manera estructurada a lo largo del CVDS, se logra un sistema que no solo es funcional, sino también bien diseñado, documentado y mantenible, lo cual es esencial para la longevidad y la evolución de cualquier aplicación.

---

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

**Información general**

| | | | |
|---|---|---|---|
| **Fecha de creación** | 26/11/2025 | **Dirección** | DIRECCIÓN DE PERACIONES |

| | |
|---|---|
| **Escuela** | 26/11/2025 |
| **Programa** | INGENIERIA DE SISTEMAS |

**Identificadores del curso**

| | |
|---|---|
| **Malla:** | VG002 |
| **Código:** | DIS07 |
| **Asignatura** | Programación Orientada a Objetos 1 |
| **Semestre (Nivel):** | 2 |
| **Temática:** | 8 |

**Tema: Aplicación de Fundamentos POO en Proyectos Básicos**

**1) Definición**

La Aplicación de Fundamentos POO en Proyectos Básicos es la fase de síntesis práctica donde los conceptos teóricos (clases, objetos, encapsulamiento, modularidad) se traducen en código funcional para resolver problemas simples. Se centra en el desarrollo de aplicaciones de consola que modelan sistemas básicos, como simulaciones de inventarios o calculadoras, utilizando un enfoque estricto de orientación a objetos para asegurar que cada parte del sistema tenga una responsabilidad clara.

## 2) Conceptos Clave

A. Desarrollo de Aplicaciones de Consola Orientadas a Objetos

Una Aplicación de Consola es un programa que interactúa con el usuario mediante texto (entrada y salida).

- Enfoque POO: En lugar de utilizar una secuencia lineal de funciones como en la programación estructurada, el enfoque POO en la consola utiliza clases (Ej. Menu, ServicioDeDatos) para organizar la lógica. El programa principal (main) simplemente orquesta la interacción entre estos objetos.

B. Simulación de Sistemas Básicos (CRUD Estático)

La simulación de sistemas básicos, como un inventario o una calculadora, es fundamental para practicar la POO.

- CRUD Estático: CRUD (*Create, Read, Update, Delete*) hace referencia a las cuatro operaciones básicas de la persistencia de datos. En proyectos básicos, se habla de CRUD estático cuando la información se almacena temporalmente en estructuras de datos en memoria (Ej. *Arrays* o Listas de la Unidad 4) y se pierde al finalizar la ejecución del programa, permitiendo al desarrollador concentrarse en el modelado y la lógica orientada a objetos, y no en la conexión a bases de datos.

C. Integración de Encapsulamiento y Modularidad

Los proyectos básicos deben actuar como laboratorios para integrar los principios de diseño:

- Encapsulamiento: Se aplica rigurosamente al declarar atributos como private y proporcionar acceso a través de métodos públicos (getters y setters). Esto asegura que el estado de los objetos (Ej. el stock de un producto) solo pueda ser alterado por lógica interna controlada.
- Modularidad: El código del proyecto se organiza en paquetes que separan responsabilidades (Ej. un paquete para las Clases Modelo (Producto) y otro para las Clases de Servicio (GestionInventario)). Esto facilita que cada clase se enfoque en una única tarea.

---

## 3) Ejemplos en Texto

Diseñemos un proyecto básico de Simulación de Inventario de una Tienda de Electrónica en consola.

1. Definición de Clases y Encapsulamiento:
    - Se define la Clase Modelo Producto con atributos privados (id, nombre, precio, stock).
    - Se implementan métodos públicos getNombre() y setStock(int nuevoStock). El método setStock incluye la lógica de validación para evitar que el *stock* sea un número negativo, encapsulando y protegiendo el dato.
2. Modularidad y Clases de Servicio:
    - Se crea una Clase de Servicio GestionInventario cuyo único propósito es manejar la lista de objetos Producto.
    - Esta clase contiene métodos como crearProducto(Producto p), buscarProducto(int id) y actualizarStock(int id, int cantidad). El menú principal solo interactúa con esta clase de servicio, manteniendo la modularidad y separación de la lógica.
3. Simulación CRUD Estático:
    - La clase GestionInventario utiliza una ArrayList<Producto> (Lista) para almacenar todos los objetos Producto.
    - Cuando el usuario ejecuta la aplicación y selecciona la opción "Crear Producto", se instancia un objeto Producto y se añade a la lista (operación Create).
    - Cuando el usuario selecciona "Ver Inventario", el sistema recorre la lista y muestra los detalles (operación Read).
    - La aplicación simula las operaciones sin depender de un archivo o base de datos externa, enfocándose en cómo los objetos POO interactúan entre sí.

---

## 4) Conclusiones

La Aplicación de Fundamentos POO en Proyectos Básicos es el paso final para consolidar la comprensión del paradigma, ya que obliga a los estudiantes a aplicar activamente la teoría. Estos proyectos son la prueba de fuego que demuestra el dominio de la Clase y el Objeto como unidades de modelado. Al desarrollar sistemas sencillos (CRUD estático), se refuerza la importancia del encapsulamiento para la integridad de los datos y de la modularidad para la organización del código, preparando al desarrollador para abordar sistemas más grandes y complejos en el futuro.

---

## 5) Bibliografía


- Aristizábal Martínez, D. A. & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos*: (1 ed.). Instituto Tecnológico Metropolitano. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/253798
- López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*: ( ed.). Grupo Editorial Patria. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/39461
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*: ( ed.). RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=15
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*: ( ed.). Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/70431
- Ruiz Rodríguez, R. (2009). *Fundamentos de la programación orientada a objetos: una aplicación a las estructuras de datos en Java*: ( ed.). El Cid Editor. https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/34869

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 1 |

**tema: Principios Avanzados de Programación Orientada a Objetos**

---

### 1. Definición

Los principios avanzados de la Programación Orientada a Objetos (POO) son un conjunto de buenas prácticas y conceptos que buscan maximizar la cohesión, minimizar el acoplamiento y fomentar la reutilización de código en aplicaciones de software complejas. Estos principios complementan los fundamentos de la POO —como encapsulamiento, herencia y polimorfismo— introduciendo enfoques más refinados que promueven la mantenibilidad, escalabilidad y legibilidad del software (Hernández & Baquero, 2023). Además, estos principios son esenciales en la construcción de arquitecturas robustas y en la implementación de patrones de diseño.

La cohesión, el acoplamiento y la reutilización no solo son métricas conceptuales de calidad del software, sino también criterios que guían la toma de decisiones arquitectónicas y de diseño. En entornos profesionales, comprender y aplicar estos principios es clave para entregar software confiable y evolutivo (Vázquez, 2020).

---

### 2. Conceptos clave

- **Cohesión**: Se refiere al grado en que los elementos de un módulo o clase están relacionados con la funcionalidad que proporciona. Una clase altamente cohesionada concentra sus métodos y atributos en torno a una sola responsabilidad, lo cual facilita su comprensión, prueba y modificación (Hernández & Baquero, 2023).

- **Acoplamiento**: Representa la dependencia entre módulos o clases. Se busca un acoplamiento bajo, es decir, que las clases sean lo más independientes posible, para minimizar el impacto de cambios en otras partes del sistema. Un alto acoplamiento es generalmente un signo de un diseño rígido y frágil (Vázquez, 2020).

- **Reutilización de código**: Es la capacidad de emplear componentes de software existentes en nuevos contextos, reduciendo la duplicidad y los errores. En la POO, esto se logra mediante herencia, composición, interfaces y la aplicación de principios como la inversión de dependencias y la segregación de interfaces (González & Gallo, 2021).

- **Principios SOLID**: Aunque no explícitamente enunciados en el syllabus, los principios SOLID (Responsabilidad Única, Abierto/Cerrado, Sustitución de Liskov, Segregación de Interfaces e Inversión de Dependencias) encapsulan estos conceptos y son la base teórica para alcanzar una alta cohesión, bajo acoplamiento y reutilización efectiva (Cairo Battistutti, 2022).

- **Manejo de excepciones en entornos orientados a objetos**: Es la implementación sistemática de mecanismos para capturar y tratar errores en tiempo de ejecución sin comprometer la estabilidad del sistema. Un buen manejo de excepciones también contribuye a un acoplamiento controlado y a mantener la cohesión en las clases (Navarro & Rodríguez, 2023).

---

**3. Ejemplos en texto**

- ***Ejemplo 1: Cohesión alta en una clase Factura***

  *Una clase Factura con métodos exclusivamente relacionados con el cálculo de totales, la aplicación de descuentos y la generación del resumen en PDF refleja una cohesión alta. Si además se delega la persistencia a un repositorio externo, se evita mezclar responsabilidades.*

- ***Ejemplo 2: Bajo acoplamiento entre módulos***

  *En una aplicación web, el módulo de autenticación no debería depender directamente del módulo de base de datos, sino de una interfaz. Esto permite cambiar la implementación de almacenamiento (por ejemplo, de SQL a MongoDB) sin impactar la lógica de autenticación.*

- ***Ejemplo 3: Reutilización de una clase Logger***

  *Una clase Logger bien diseñada, que implementa una interfaz estándar para registrar mensajes, puede ser utilizada en múltiples aplicaciones (escritorio, web, móvil) sin modificaciones.*

- ***Ejemplo 4: Manejo de excepciones***

  *Una clase ProcesadorDePagos podría capturar excepciones específicas, como PagoRechazadoException, para ofrecer retroalimentación al usuario y registrar el incidente sin interrumpir el flujo de la aplicación.*

---

**4. Conclusión**

Los principios avanzados de la Programación Orientada a Objetos representan el paso natural para quienes buscan diseñar y construir software profesionalmente. La cohesión alta y el bajo acoplamiento aumentan la modularidad y facilitan el mantenimiento del sistema, mientras que la reutilización de código permite optimizar el desarrollo, reduciendo costos y tiempos. El manejo adecuado de excepciones, además de mejorar la experiencia del usuario final, refuerza la estabilidad de las aplicaciones. Estos principios no solo impactan en la calidad técnica del producto, sino también en la capacidad de los equipos de ingeniería para adaptarse a los cambios y evolucionar las soluciones.

El dominio de estos principios es una competencia transversal a toda la ingeniería de software y constituye una base indispensable para avanzar hacia temas más complejos como patrones de diseño, arquitectura limpia y metodologías ágiles.

---

**5. Referencias (APA 7)**


- Cairo Battistutti, O. (2022). Aprende a programar en Java: De cero al infinito.
- González, A. H., & Gallo, S. L. (2021). Herencia y polimorfismo.
- Hernández, M., & Baquero, L. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.
- Navarro, D., & Rodríguez, S. (2023). Programación orientada a objetos en Python: Teoría, patrones y prácticas avanzadas. Editorial Alfaomega.
- Vázquez, M. (2020). Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 2 |

**tema: Interfaces y Clases Abstractas**

---

### 1. Definición

Las **interfaces** y las **clases abstractas** son mecanismos fundamentales en la Programación Orientada a Objetos (POO) para definir contratos, promover la reutilización y facilitar la extensión del comportamiento de los sistemas de software. Ambos proporcionan un marco para el diseño basado en abstracciones, esencial para alcanzar flexibilidad, cohesión y bajo acoplamiento en las aplicaciones (Garay Avendaño, 2022).

Una **clase abstracta** es una clase que no puede ser instanciada directamente y que puede contener tanto métodos abstractos (sin implementación) como métodos concretos (con implementación). Por su parte, una **interface** define un conjunto de métodos que una clase concreta debe implementar, pero no puede incluir lógica implementada (en algunos lenguajes modernos como Java, las interfaces pueden contener métodos con implementación predeterminada mediante default methods) (Gil Prado, 2023).

El uso adecuado de interfaces y clases abstractas permite diseñar sistemas extensibles y desacoplados, siguiendo principios como la inversión de dependencias y el principio de abierto/cerrado (Hernández & Baquero, 2023).

---

**2. Conceptos clave**

- Clases abstractas: Plantillas que permiten definir atributos y comportamientos generales que serán heredados por las subclases. Permiten incluir tanto métodos abstractos como concretos y campos con estado (atributos). Son útiles cuando varias clases comparten comportamiento parcial (Cairo Battistutti, 2022).

- Interfaces: Definen un contrato que las clases implementadoras deben cumplir, sin preocuparse por cómo implementan los métodos. Las interfaces no pueden contener estado (atributos) salvo constantes, y son ideales para modelar capacidades comunes entre clases no relacionadas jerárquicamente (Garay Avendaño, 2022).

- Diseño orientado a interfaces: En lugar de depender de clases concretas, los módulos dependen de interfaces, lo cual facilita cambiar las implementaciones sin alterar el código cliente, favoreciendo el bajo acoplamiento (Vázquez, 2020).

- Dependencia inversa: Un principio del diseño que establece que las clases de alto nivel no deben depender de clases concretas, sino de abstracciones (interfaces o clases abstractas), promoviendo la flexibilidad y la mantenibilidad (Hernández & Baquero, 2023).

- Interfaces funcionales: Interfaces con un solo método abstracto, usadas ampliamente en programación funcional, como en expresiones lambda y streams en Java 8+ (Gil Prado, 2023).

---

**3. Ejemplos en texto**

- **Ejemplo 1**: Clase abstracta Animal

  Una clase abstracta Animal puede definir métodos concretos como respirar() y métodos abstractos como hacerSonido(). Subclases como Perro o Gato heredan la implementación de respirar() pero deben implementar hacerSonido() de acuerdo a su especie.

- **Ejemplo 2**: Interface Pagable

  Una interface Pagable define el método pagar(). Clases dispares como Factura y Nómina implementan esta interface para garantizar que ambas sean "pagables", sin compartir una relación jerárquica.

- **Ejemplo 3**: Diseño orientado a interfaces

  En un sistema de envío de mensajes, una clase Notificador puede depender de una interface CanalDeEnvio en lugar de una clase concreta Email. De esta forma, se pueden agregar implementaciones como SMS o Push sin cambiar Notificador.

- **Ejemplo 4**: Interface funcional

En Java 8, la interface funcional Comparator<T> permite usar expresiones lambda para definir criterios de ordenación dinámicamente, como en list.sort((a,b) -> a.getEdad() - b.getEdad()).

---

**4. Conclusión**

Las interfaces y las clases abstractas son piedras angulares en el diseño de sistemas orientados a objetos, ya que permiten separar el "qué" del "cómo" y promueven la reutilización y flexibilidad. Mientras las clases abstractas son útiles para compartir comportamiento común en una jerarquía, las interfaces son ideales para definir contratos entre clases heterogéneas. La aplicación adecuada de estas abstracciones permite cumplir con principios fundamentales de diseño como el bajo acoplamiento, la alta cohesión y la inversión de dependencias, todos cruciales en sistemas escalables y mantenibles.

El dominio de estos conceptos permite a los profesionales de software diseñar sistemas robustos y adaptables a cambios, lo cual es un objetivo clave en el desarrollo moderno.

---

**5. Referencias (APA 7)**


- Cairo Battistutti, O. (2022). *Aprende a programar en Java: De cero al infinito*.
- Garay Avendaño, F. (2022). *Lenguaje de programación orientado a objetos: Introducción a POO, fases de diseño y construcción con lenguajes de cuarta generación*.
- Gil Prado, G. E. (2023). *Interfaces*.
- Hernández, M., & Baquero, L. (2023). *Programación orientada a objetos en Java: Buenas prácticas*. Ediciones de la U.
- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 3 |

**tema: Estructuras de Datos Avanzadas**

---

### 1. Definición

Las **estructuras de datos avanzadas** en Programación Orientada a Objetos (POO) son colecciones y mecanismos especializados para almacenar, organizar y manipular datos de forma eficiente, encapsulados dentro de objetos. Estas estructuras permiten a los programas administrar grandes volúmenes de información y facilitar operaciones complejas, manteniendo la abstracción y modularidad propias de la POO (García & Mendoza, 2022).

En el contexto de la POO, las estructuras de datos tradicionales como listas, pilas y colas se implementan mediante clases y objetos, lo que permite su personalización y extensión. Además, las colecciones genéricas introducidas en lenguajes como Java y C# han potenciado la reutilización segura de código, permitiendo operar con cualquier tipo de dato sin comprometer la integridad del tipo (Vázquez, 2020).

---

### 2. Conceptos clave

- **Listas**: Colecciones lineales de elementos en las que el orden de inserción se respeta. En POO, se implementan mediante clases como ArrayList o

LinkedList, cada una con características particulares (García & Mendoza, 2022).

- **Pilas (Stacks)**: Estructuras de tipo LIFO (*Last In, First Out*) que permiten apilar y desapilar elementos. Se utilizan en la implementación de algoritmos recursivos, control de llamadas o deshacer acciones.
- **Colas (Queues)**: Estructuras FIFO (*First In, First Out*) útiles para gestionar procesos en espera, tareas de impresión o mensajes en sistemas distribuidos. Variantes como PriorityQueue permiten ordenamientos adicionales.
- **Conjuntos (Sets)**: Colecciones que no permiten duplicados. Implementaciones como HashSet o TreeSet ofrecen eficiencia en las operaciones de búsqueda y manipulación de elementos únicos (Hernández & Baquero, 2023).
- **Mapas (Maps)**: Asociaciones clave-valor que permiten almacenar pares de datos. Ejemplos comunes incluyen HashMap, TreeMap y LinkedHashMap, fundamentales para implementar diccionarios, índices y caches (García & Mendoza, 2022).
- **Colecciones genéricas**: Permiten trabajar con estructuras de datos parametrizadas por tipo, garantizando la seguridad en tiempo de compilación al evitar errores de tipo en tiempo de ejecución (Vázquez, 2020).
- **Comparación de estructuras**: La elección entre estructuras tradicionales y colecciones modernas depende de factores como la eficiencia, la legibilidad y la compatibilidad con algoritmos avanzados.

---

**3. Ejemplos en texto**

- **Ejemplo 1: Uso de una ArrayList para gestionar productos**

En una tienda virtual, se puede utilizar un ArrayList<Producto> para almacenar los productos disponibles en inventario, ya que permite acceder a los elementos por índice y ajustar dinámicamente su tamaño.

- **Ejemplo 2: Implementación de una pila para control de navegación**

En un navegador web, una Stack<String> guarda las páginas visitadas para implementar la funcionalidad de "volver atrás", desapilando las páginas en orden inverso al que fueron visitadas.

- **Ejemplo 3: Cola de impresión con prioridad**

Un PriorityQueue<Documento> puede ordenar los documentos en espera según su urgencia, asegurando que los documentos más importantes se impriman primero.

- **Ejemplo 4: Mapa para asociar usuarios a sesiones**

En una aplicación web, un HashMap<String, Sesion> puede utilizarse para relacionar el identificador del usuario con su objeto de sesión activo, facilitando la gestión eficiente de usuarios conectados.

## 4. Conclusión

El uso de estructuras de datos avanzadas en la POO permite a los desarrolladores crear aplicaciones más robustas y eficientes. Elegir la estructura correcta según el problema específico es esencial para optimizar recursos, mejorar la legibilidad del código y garantizar un comportamiento predecible. La combinación de estas estructuras con los principios de la POO, como el encapsulamiento y la reutilización, contribuye a diseñar software modular y escalable, adaptado a las demandas actuales de procesamiento de datos en tiempo real y gran escala.

Dominar estas estructuras es indispensable para los profesionales del desarrollo de software, pues constituyen la base sobre la cual se construyen algoritmos sofisticados y se gestionan procesos concurrentes y distribuidos.

---

## 5. Referencias (APA 7)


- García, P., & Mendoza, J. (2022). *Introducción a la persistencia de datos con Hibernate y JPA en Java*. Ediciones RA-MA.

- Hernández, M., & Baquero, L. (2023). *Programación orientada a objetos en Java: Buenas prácticas*. Ediciones de la U.

- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 4 |

**tema: Patrones de Diseño Orientados a Objetos**

---

### 1. Definición

Los **patrones de diseño** son soluciones reutilizables y probadas a problemas comunes que surgen durante el desarrollo de software orientado a objetos. Un patrón de diseño no es un fragmento de código específico, sino una guía o esquema para estructurar las clases y objetos de una manera que resuelva un problema recurrente dentro de un contexto determinado (Vázquez, 2020).

Estos patrones encapsulan las mejores prácticas de ingenieros experimentados y promueven principios fundamentales como la reutilización, el bajo acoplamiento, la cohesión y la mantenibilidad. Existen diversas clasificaciones, pero las más difundidas son las de **creacionales**, **estructurales** y **de comportamiento**, propuestas inicialmente por el grupo conocido como los "Gang of Four" (GoF) (Ruiz & López, 2023).

---

### 2. Conceptos clave

- **Propósito de los patrones de diseño**: Facilitan el diseño robusto y flexible, minimizando la complejidad y mejorando la capacidad de evolución de los

sistemas. Actúan como un lenguaje común entre los desarrolladores para comunicar soluciones de alto nivel (Vázquez, 2020).

- **Clasificación de patrones**:
  - **Creacionales**: Se centran en la instanciación controlada de objetos, abstraen el proceso de creación para desacoplar la clase cliente de las clases concretas.
  - **Estructurales**: Describen cómo organizar clases y objetos para formar estructuras más grandes y flexibles.
  - **De comportamiento**: Se enfocan en la interacción y responsabilidad entre objetos para cumplir tareas específicas (Ruiz & López, 2023).
- **Patrones creacionales**:
  - **Singleton**: Garantiza que una clase tenga solo una instancia global y proporciona un punto de acceso a ella.
  - **Factory Method**: Define una interfaz para crear objetos, dejando a las subclases la decisión de qué clase instanciar.
  - **Builder**: Separa la construcción de un objeto complejo de su representación final (Vázquez, 2020).
- **Patrones estructurales**:
  - **Adapter**: Permite la colaboración entre interfaces incompatibles.
  - **Composite**: Organiza objetos en estructuras jerárquicas para tratarlos de manera uniforme.
  - **Decorator**: Agrega responsabilidades dinámicamente a un objeto sin alterar su estructura interna.
- **Patrones de comportamiento**:
  - **Observer**: Define una relación de dependencia uno-a-muchos para que, cuando un objeto cambie de estado, sus dependientes sean notificados.
  - **Strategy**: Define una familia de algoritmos, encapsulándolos y haciéndolos intercambiables en tiempo de ejecución (Ruiz & López, 2023).

---

**3. Ejemplos en texto**

- **Ejemplo 1: Singleton en un sistema de configuración**

Una clase ConfiguracionSistema implementa el patrón Singleton para garantizar que toda la aplicación acceda a una única instancia que contiene los parámetros de configuración cargados desde un archivo externo.

- **Ejemplo 2: Factory Method para crear usuarios**

Una clase UsuarioFactory decide qué tipo concreto de objeto Usuario devolver (Admin, Cliente, Invitado) según las credenciales del usuario al inicio de sesión.

- **Ejemplo 3: Decorator para componentes gráficos**

En una interfaz gráfica, el patrón Decorator permite añadir bordes, sombras o scrollbars a un componente Ventana sin modificar su código base.

- **Ejemplo 4: Observer en un sistema de notificaciones**

En una aplicación de bolsa de valores, los observadores Inversor se suscriben al objeto Accion para recibir notificaciones inmediatas cuando el precio cambia.

- **Ejemplo 5: Strategy en un motor de cálculo de impuestos**

Un sistema contable permite seleccionar en tiempo de ejecución diferentes estrategias de cálculo de impuestos dependiendo del país o región.

---

**4. Conclusión**

Los patrones de diseño orientados a objetos son esenciales para el desarrollo profesional de software, ya que permiten aplicar soluciones efectivas y robustas a problemas recurrentes. Su comprensión y correcta aplicación promueve diseños flexibles, extensibles y fáciles de mantener. Además, fomentan un lenguaje común entre los desarrolladores, lo que facilita la colaboración en proyectos complejos.

En un contexto profesional, dominar estos patrones capacita a los ingenieros para diseñar arquitecturas limpias y adaptables a los cambios inevitables en los requisitos de negocio y tecnología. Por ello, constituyen una competencia indispensable en la formación de los desarrolladores.

---

**5. Referencias (APA 7)**


- Ruiz, E., & López, C. (2023). *Creación de interfaces gráficas modernas con JavaFX y patrones de diseño orientados a objetos*. Editorial Síntesis.

- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 5 |

**tema: Desarrollo de Interfaces Gráficas con Orientación a Objetos**

---

### 1. Definición

El **desarrollo de interfaces gráficas de usuario (GUI)** en el contexto de la Programación Orientada a Objetos (POO) consiste en crear aplicaciones interactivas que permiten al usuario comunicarse con el sistema mediante componentes visuales como ventanas, botones, menús, cuadros de texto, entre otros. Este enfoque se basa en el principio de encapsular la lógica de negocio y la presentación en componentes independientes, pero interrelacionados (Carrillo & Navarro, 2023).

En entornos modernos, las interfaces gráficas se implementan mediante bibliotecas y frameworks especializados, como JavaFX, Swing o PyQt, que siguen el paradigma orientado a objetos, permitiendo heredar, componer y personalizar componentes visuales. La POO facilita la reutilización de componentes gráficos y la separación de responsabilidades, garantizando la mantenibilidad y la escalabilidad del software (Oviedo Regino, 2015).

---

### 2. Conceptos clave

- **Componentes gráficos**: Objetos predefinidos que representan elementos visuales como botones (Button), cuadros de texto (TextField), etiquetas (Label), etc. Cada componente es una clase que puede ser extendida o compuesta para crear nuevos elementos personalizados (Carrillo & Navarro, 2023).

- **Eventos y manejadores**: Mecanismo que permite a los objetos responder a acciones del usuario (clics, teclas, movimientos). La programación orientada a eventos implementa métodos de escucha (listeners) para reaccionar ante estos eventos (Moreno Pérez, 2015).

- **Diseño de ventanas interactivas**: Consiste en definir la estructura jerárquica de componentes gráficos dentro de una ventana o Stage, organizados mediante paneles (Pane, GridPane, VBox) que gestionan la distribución espacial.

- **Vinculación con la lógica de negocio**: Se refiere a la integración de la GUI con los procesos internos de la aplicación, garantizando que las acciones en la interfaz actualicen o consulten los datos del sistema sin romper el encapsulamiento.

- **Personalización de estilos**: Uso de CSS o mecanismos equivalentes para modificar la apariencia de los componentes gráficos, separando el diseño visual de la lógica funcional (Ruiz & López, 2023).

- **Frameworks orientados a objetos para GUI**:
    - *JavaFX*: Framework moderno que reemplaza Swing en Java, con soporte para FXML, CSS y gráficos avanzados.
    - *Swing*: Biblioteca clásica para GUI en Java, ampliamente utilizada por su robustez y compatibilidad.
    - *PyQt*: Toolkit para crear GUIs en Python, basado en Qt, que permite crear aplicaciones multiplataforma.

---

**3. Ejemplos en texto**

- **Ejemplo 1: Ventana de inicio de sesión en JavaFX**

Una clase LoginWindow hereda de Stage y contiene objetos TextField para el usuario, PasswordField para la contraseña y un Button para enviar los datos. Los eventos del botón invocan la lógica de autenticación encapsulada en la clase AuthService.

- **Ejemplo 2: Manejo de eventos en Swing**

Un objeto JButton llamado btnGuardar en Swing registra un ActionListener que define qué acciones ejecutar cuando el usuario haga clic, como validar los campos y persistir los datos en la base.

- **Ejemplo 3: Personalización con CSS**

En una aplicación JavaFX, los estilos visuales de los botones y cuadros de texto se definen en un archivo style.css, permitiendo cambiar colores, bordes y fuentes sin modificar las clases Java.

- **Ejemplo 4: PyQt con herencia**

Una clase MainWindow en Python hereda de QMainWindow y define métodos que organizan menús, barras de herramientas y contenidos centrales, demostrando la flexibilidad de la herencia en GUI.

---

**4. Conclusión**

El desarrollo de interfaces gráficas con orientación a objetos es esencial para la construcción de aplicaciones centradas en el usuario. La POO facilita la creación modular de componentes visuales, su personalización y su integración con la lógica interna, garantizando un diseño limpio y mantenible. Además, los frameworks actuales proporcionan herramientas que siguen principios sólidos de diseño, lo que permite a los desarrolladores construir GUIs modernas, adaptables y consistentes.

Dominar estas técnicas permite a los profesionales ofrecer experiencias de usuario atractivas y funcionales, diferenciando sus aplicaciones en un mercado altamente competitivo.

---

**5. Referencias (APA 7)**


- Carrillo, P., & Navarro, J. (2023). *Desarrollo de interfaces gráficas con orientación a objetos: Principios y prácticas con JavaFX*. Ediciones Pirámide.
- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=1
- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/70431?page=1
- Ruiz, E., & López, C. (2023). *Creación de interfaces gráficas modernas con JavaFX y patrones de diseño orientados a objetos*. Editorial Síntesis.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 6 |

**tema: Persistencia de Datos en Aplicaciones Orientadas a Objetos**

---

### 1. Definición

La **persistencia de datos** en aplicaciones orientadas a objetos hace referencia a la capacidad de almacenar y recuperar el estado de los objetos más allá de la duración del programa, es decir, mantener su información en un medio permanente, como bases de datos o archivos, para que pueda ser reutilizada en futuras ejecuciones (García & Mendoza, 2022).

En la POO, la persistencia representa un desafío, ya que los objetos encapsulan datos y comportamientos en estructuras complejas que no siempre son triviales de mapear a formatos de almacenamiento plano. Para resolver esta brecha, se utilizan técnicas como el **mapeo objeto-relacional (ORM)**, así como persistencia en archivos estructurados (XML, JSON) o en bases de datos NoSQL, garantizando que la integridad de las relaciones y la cohesión interna de los objetos se mantengan intactas (Moreno Pérez, 2015).

---

### 2. Conceptos clave

- **Bases de datos relacionales y no relacionales**: Las bases relacionales (como MySQL, PostgreSQL) organizan la información en tablas y relaciones, mientras que las no relacionales (como MongoDB) manejan documentos, claves-valor o grafos, lo que se adapta mejor a ciertas estructuras de objetos complejos (García & Mendoza, 2022).

- **Mapeo Objeto-Relacional (ORM)**: Técnica que traduce automáticamente entre las clases de un lenguaje orientado a objetos y las tablas de una base de datos relacional. Frameworks como Hibernate para Java o SQLAlchemy para Python permiten esta traducción minimizando el esfuerzo manual (Hernández & Baquero, 2023).

- **Persistencia en archivos**: Alternativa ligera para almacenar datos cuando no es necesaria una base de datos completa. Se utilizan formatos estandarizados como JSON (JavaScript Object Notation) y XML (eXtensible Markup Language), que son fácilmente interpretables y compatibles con la estructura de objetos (Oviedo Regino, 2015).

- **Diseño de modelos de datos**: Implica definir clases que reflejen las entidades del dominio de negocio, sus atributos y las relaciones entre ellas, y cómo estas se corresponden con las estructuras de almacenamiento, respetando los principios de cohesión y encapsulamiento.

- **Integración lógica-persistencia**: La capa de persistencia debe integrarse con la lógica de negocio a través de repositorios o servicios, evitando acoplar directamente las clases de dominio a las API de almacenamiento (Vázquez, 2020).

---

**3. Ejemplos en texto**

- **Ejemplo 1: Persistencia con Hibernate**

En una aplicación Java, la clase Empleado se mapea a la tabla empleados mediante anotaciones JPA (@Entity, @Table, @Id). Hibernate gestiona automáticamente las operaciones CRUD, sin necesidad de escribir sentencias SQL explícitas.

- **Ejemplo 2: Archivos JSON para configuración**

Una clase Configuración serializa sus atributos en un archivo config.json para guardar preferencias del usuario. Al iniciar la aplicación, el archivo se lee y se reconstruye el objeto con su estado anterior.

- **Ejemplo 3: Base de datos NoSQL con MongoDB**

Un sistema de catálogo de productos usa una colección productos en MongoDB, donde cada documento corresponde directamente a un objeto Producto con atributos anidados, ideal para datos semi-estructurados.

- **Ejemplo 4: Capa de repositorios**

Una clase ClienteRepository actúa como intermediaria entre la base de datos y la lógica de negocio, encapsulando las operaciones de almacenamiento y recuperación

y evitando que la clase ClienteService dependa directamente del mecanismo de persistencia.

---

**4. Conclusión**

La persistencia de datos es un aspecto esencial de las aplicaciones orientadas a objetos, ya que garantiza la continuidad de la información a través del tiempo y permite construir sistemas confiables y consistentes. La utilización de técnicas y herramientas adecuadas como ORM, bases NoSQL o archivos estructurados, junto con un diseño que respete la separación de responsabilidades, permite a los desarrolladores mantener la integridad y cohesión de sus modelos de dominio.

El dominio de estas técnicas posiciona al profesional como un desarrollador capaz de construir sistemas robustos, escalables y preparados para integrarse en entornos empresariales complejos.

---

**5. Referencias (APA 7)**


- García, P., & Mendoza, J. (2022). *Introducción a la persistencia de datos con Hibernate y JPA en Java*. Ediciones RA-MA.

- Hernández, M., & Baquero, L. (2023). *Programación orientada a objetos en Java: Buenas prácticas*. Ediciones de la U.

- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=1

- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/70431?page=1

- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 7 |

**tema: Conexión y Consumo de APIs**

---

### 1. Definición

Las **APIs (Application Programming Interfaces)** son mecanismos que permiten que dos sistemas de software diferentes se comuniquen entre sí mediante un conjunto definido de reglas y protocolos. En el contexto de la Programación Orientada a Objetos (POO), la conexión y consumo de APIs consiste en integrar aplicaciones orientadas a objetos con servicios externos, encapsulando la lógica de comunicación y asegurando que los objetos puedan enviar y recibir datos de manera transparente y eficiente (Navarro & Rodríguez, 2023).

El consumo de **APIs RESTful**, que se basa en el protocolo HTTP y sigue los principios de la arquitectura REST, es una práctica común para permitir que las aplicaciones intercambien datos en formatos estandarizados como JSON o XML. La POO facilita esta tarea al modelar las peticiones y las respuestas como objetos, lo que permite mantener el encapsulamiento, la cohesión y el bajo acoplamiento en el diseño (Oviedo Regino, 2015).

---

### 2. Conceptos clave

- **API RESTful**: Un estilo arquitectónico que define cómo deben diseñarse las interfaces para que sean escalables, uniformes y basadas en recursos accesibles mediante los métodos HTTP estándar: GET, POST, PUT, DELETE (Navarro & Rodríguez, 2023).
- **Solicitudes HTTP**: Los objetos en la POO generan y envían solicitudes HTTP a los endpoints del API, encapsulando detalles como cabeceras, cuerpo de la petición y parámetros.
- **Respuesta HTTP**: La respuesta recibida del servidor se transforma en un objeto o estructura de datos que la aplicación cliente puede manipular fácilmente.
- **Autenticación en APIs**: Los mecanismos de seguridad, como OAuth 2.0, permiten autenticar y autorizar a los usuarios y aplicaciones para acceder a los recursos expuestos por las APIs (Moreno Pérez, 2015).
- **Integración en POO**: La lógica de consumo de APIs debe implementarse en clases especializadas que separen la interacción externa de la lógica interna del sistema, aplicando principios de diseño como la inversión de dependencias y el patrón fachada para simplificar el acceso (Vázquez, 2020).

---

**3. Ejemplos en texto**

- **Ejemplo 1: Consumo de API RESTful con objetos**

Una clase ClienteAPI envía una solicitud HTTP GET a un endpoint /usuarios y recibe un JSON que se transforma en una lista de objetos Usuario, cada uno representando una fila de datos devuelta por el servidor.

- **Ejemplo 2: Encapsulamiento de autenticación OAuth 2.0**

La clase TokenService encapsula la lógica para obtener y renovar un token de acceso OAuth, separando las credenciales de usuario de la lógica de negocio que utiliza el token para interactuar con el API.

- **Ejemplo 3: Integración con servicios externos**

Una aplicación de e-commerce implementa la clase PasarelaPago para conectarse a la API de un proveedor de pagos externo, enviando la información de la transacción y recibiendo un objeto ConfirmacionPago.

- **Ejemplo 4: Manejo de errores HTTP**

Una clase ApiResponseHandler encapsula la gestión de errores como códigos 404 (recurso no encontrado) o 500 (error del servidor) y genera excepciones específicas que pueden ser capturadas por la lógica de la aplicación.

---

**4. Conclusión**

La conexión y consumo de APIs son prácticas esenciales en el desarrollo de aplicaciones modernas, ya que permiten integrar servicios distribuidos y aprovechar

funcionalidades externas sin reinventar soluciones. La POO aporta las herramientas conceptuales y técnicas para encapsular la lógica de comunicación, mantener el bajo acoplamiento y diseñar sistemas que sean robustos frente a cambios en los servicios externos.

El dominio de estas prácticas permite a los profesionales diseñar aplicaciones interoperables y escalables, una habilidad clave en entornos donde la integración de servicios en la nube y microservicios es la norma.

---

**5. Referencias (APA 7)**


- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=1

- Navarro, D., & Rodríguez, S. (2023). *Programación orientada a objetos en Python: Teoría, patrones y prácticas avanzadas*. Editorial Alfaomega.

- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/70431?page=1

- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.

Corporación Unificada Nacional de Educación Superior
VIGILADA MINEDUCACIÓN

## INFORMACIÓN GENERAL

| | | | |
|---|---|---|---|
| Fecha de creación | 17-07-2025 | Dirección: | DIRECCIÓN DE OPERACIONES |

| | |
|---|---|
| Escuela | INGENIERIAS |
| Programa | INGENIERIA DE SISTEMAS |

## IDENTIFICACIÓN DEL CURSO

| | |
|---|---|
| Malla: | VG002 |
| Código: | DIS11 |
| Asignatura: | PROGRAMACION ORIENTADA A OBJETOS 2 |
| Semestre (Nivel) | 3 |
| Temática: | 8 |

**tema: Pruebas y Depuración de Aplicaciones Orientadas a Objetos**

---

### 1. Definición

Las **pruebas** y la **depuración** son actividades fundamentales en el ciclo de vida del desarrollo de software orientado a objetos. Las pruebas garantizan que las clases y objetos implementados cumplen con sus especificaciones y funcionan correctamente, mientras que la depuración es el proceso sistemático para identificar, diagnosticar y corregir errores (bugs) en la lógica de la aplicación (Oviedo Regino, 2015).

En la POO, las pruebas se estructuran comúnmente como **pruebas unitarias**, las cuales verifican de manera aislada el comportamiento de métodos y clases. Herramientas como JUnit para Java y PyTest para Python facilitan la creación de baterías de pruebas automatizadas que pueden ejecutarse continuamente para asegurar la calidad del software (Hernández & Baquero, 2023).

La depuración, por su parte, se apoya en técnicas como puntos de interrupción, inspección de estados de los objetos y seguimiento del flujo de ejecución para localizar defectos en tiempo de desarrollo (Vázquez, 2020).

---

**2. Conceptos clave**

- **Pruebas unitarias**: Validan individualmente las unidades mínimas de código (métodos o clases) para verificar que funcionan como se espera, en aislamiento del resto del sistema.
- **Cobertura de pruebas**: Medida del grado en que las pruebas ejercitan el código de la aplicación. Un nivel alto de cobertura reduce el riesgo de errores no detectados.
- **Mocking y stubbing**: Técnicas para simular el comportamiento de dependencias externas o complejas durante las pruebas, facilitando el aislamiento de las unidades bajo prueba (Navarro & Rodríguez, 2023).
- **Automatización de pruebas**: La ejecución automática de las pruebas unitarias a través de frameworks asegura retroalimentación rápida y continua en cada cambio del código.
- **Depuración en POO**: Uso de herramientas integradas (como los depuradores de IDEs) para examinar en tiempo real el estado interno de los objetos, las pilas de llamadas y los flujos de control, detectando y corrigiendo defectos lógicos (Moreno Pérez, 2015).
- **Pruebas en clases y métodos**: La modularidad de la POO facilita probar cada clase y método de forma aislada, permitiendo un diagnóstico preciso de errores.

---

**3. Ejemplos en texto**

- **Ejemplo 1: Prueba unitaria de una clase Calculadora**

Una prueba unitaria implementada en JUnit valida que el método sumar() de la clase Calculadora devuelve el resultado correcto para diferentes combinaciones de números positivos y negativos.

- **Ejemplo 2: Mock de un repositorio de datos**

Durante la prueba de la clase ServicioDePedidos, un objeto RepositorioMock simula el comportamiento de la base de datos para no depender de una conexión real, permitiendo probar la lógica de negocio en aislamiento.

- **Ejemplo 3: Depuración con puntos de interrupción**

Al detectar que la clase ProcesadorDePagos genera resultados incorrectos, el desarrollador coloca puntos de interrupción en el método procesar() y revisa los valores internos de los objetos involucrados para encontrar el error.

- **Ejemplo 4: Integración de pruebas automatizadas**

En un proyecto Python, PyTest ejecuta automáticamente todas las pruebas cada vez que el código es modificado, asegurando que los cambios no introduzcan errores regresivos.

---

## 4. Conclusión

Las pruebas y la depuración son pilares esenciales para garantizar la calidad y confiabilidad del software orientado a objetos. La POO favorece la modularidad, lo que permite realizar pruebas precisas y eficaces en unidades aisladas y depurar con mayor facilidad gracias a la encapsulación y claridad en las responsabilidades de cada clase.

La adopción de herramientas y metodologías modernas para pruebas unitarias, automatización y depuración sistemática permite a los desarrolladores entregar software más robusto, reducir costos de mantenimiento y aumentar la satisfacción del usuario final. En el entorno profesional, estas prácticas son una competencia básica para cualquier ingeniero de software comprometido con la calidad.

---

## 5. Referencias (APA 7)


- Hernández, M., & Baquero, L. (2023). *Programación orientada a objetos en Java: Buenas prácticas*. Ediciones de la U.

- Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/106461?page=1

- Navarro, D., & Rodríguez, S. (2023). *Programación orientada a objetos en Python: Teoría, patrones y prácticas avanzadas*. Editorial Alfaomega.

- Oviedo Regino, E. M. (2015). *Lógica de programación orientada a objetos*. Ecoe Ediciones. https://elibro-net.zproxy.cun.edu.co/es/ereader/bibliocun/70431?page=1

- Vázquez, M. (2020). *Java avanzado: Aplicación de principios orientados a objetos y patrones de diseño*. Ediciones Pearson.